

CIS 5521 – Homework 2

Md. Mizanur Rahman
md.mizanur.rahman@temple.edu

September 17, 2026

Task 1

Average-Case Analysis of Binary Search

Consider Binary Search on a sorted integer array

$$A[0], A[1], \dots, A[n-1]$$

for a target value x .

In this analysis, the cost of the Binary Search is measured by the number of iterations for examining the array elements. And this is a suitable indicator since each of the iteration performs a constant amount of work and reduces the remaining search interval by approximately half.

For an array of size n , there are $n + 1$ possible outcomes of a search because the possibility of finding target elements is n at any n position and also the possibility of not finding the target elements is 1. So the total number of outcomes are $n + 1$.

For the average-case analysis, I assume that all of these $n + 1$ **outcomes are equally likely**. Therefore, the probability that the target appears at any particular position i is

$$P(x = A[i]) = \frac{1}{n+1}, \quad 0 \leq i < n.$$

The probability that the target is not present in the array is also

$$P(x \notin A) = \frac{1}{n+1}.$$

These probabilities form a valid probability distribution because

$$\sum_{i=0}^{n-1} P(x = A[i]) + P(x \notin A) = n \left(\frac{1}{n+1} \right) + \frac{1}{n+1}.$$

Therefore,

$$\frac{n}{n+1} + \frac{1}{n+1} = 1.$$

Thus, each of the $n + 1$ possible outcomes has probability

$$p_a = \frac{1}{n+1}.$$

This probability distribution is an assumption for the average-case analysis; it is not a property of the Binary Search algorithm itself.

Cost of Binary Search:

Initially, the search interval contains n elements. After one unsuccessful comparison, at most approximately

$$\frac{n}{2}$$

elements remain. After two comparisons, at most approximately

$$\frac{n}{2^2}$$

elements remain.

After k comparisons, the number of remaining candidates is approximately

$$\frac{n}{2^k}.$$

The search terminates when the remaining interval contains at most one candidate. Therefore,

$$\frac{n}{2^k} \leq 1.$$

This gives

$$n \leq 2^k.$$

Taking logarithm base 2,

$$\lg n \leq k.$$

The number of halvings needed is therefore proportional to

$$\lg n.$$

More precisely,

$$k = \Theta(\lg n).$$

Thus, every possible outcome of Binary Search requires at most logarithmic cost.

Let C_i denote the cost when the target is found at position i , and let C_F denote the cost when the target is not found.

Then, for some constants a and b ,

$$C_i \leq a \lg n + b$$

for every $0 \leq i < n$, and

$$C_F \leq a \lg n + b.$$

Expected Cost:

The average cost of the algorithm is its expected cost. Therefore,

$$E[C] = \sum_{\text{all outcomes}} P(\text{outcome})C(\text{outcome}).$$

For Binary Search, this becomes

$$E[C] = \sum_{i=0}^{n-1} P(x = A[i])C_i + P(x \notin A)C_F.$$

Since every outcome has probability

$$\frac{1}{n+1},$$

we obtain

$$E[C] = \frac{1}{n+1}C_0 + \frac{1}{n+1}C_1 + \cdots + \frac{1}{n+1}C_{n-1} + \frac{1}{n+1}C_F.$$

Factoring out the common probability,

$$E[C] = \frac{1}{n+1} (C_0 + C_1 + \cdots + C_{n-1} + C_F).$$

Since each possible cost is at most

$$a \lg n + b,$$

we have

$$E[C] \leq \frac{1}{n+1} [n(a \lg n + b) + (a \lg n + b)].$$

Combining the terms,

$$E[C] \leq \frac{1}{n+1} [(n+1)(a \lg n + b)].$$

Therefore,

$$E[C] \leq a \lg n + b.$$

Hence, the average cost of Binary Search is

$$\boxed{E[C] = O(\lg n)}.$$

Therefore, under the assumption that all n successful search positions and the one unsuccessful-search outcome are equally likely, the expected running time of Binary Search grows logarithmically with the size of the array.

Task 2

Comparison of Two Probability Models

Let

$\overline{C_S}$ = denote the average cost of a successful search

and

C_F = denote the cost of an unsuccessful search.

a) Situation 1

In the first situation, the target can appear at any one of the n array positions with probability p_a , and the probability that the target is not in the array is also p_a .

Since there are $n + 1$ equally probable cases,

$$(n + 1)p_a = 1.$$

Therefore,

$$p_a = \frac{1}{n + 1}.$$

The probability that the target is somewhere in the array is therefore

$$P(\text{target present}) = \frac{n}{n + 1}.$$

The probability that the target is absent is

$$P(\text{target absent}) = \frac{1}{n + 1}.$$

Hence, the expected cost is

$$E_1 = \frac{n}{n + 1}\overline{C}_S + \frac{1}{n + 1}C_F.$$

Therefore,

$$E_1 = \frac{n\overline{C}_S + C_F}{n + 1}.$$

b) Situation 2

In the second situation, the probability that the target is in the array is the same as the probability that the target is absent.

Therefore,

$$p_b = \frac{1}{2}.$$

When the target is present, every array position is equally likely. Therefore,

$$p_c = \frac{1}{n}.$$

Notice that p_c is a conditional probability:

$$p_c = P(\text{position } i \mid \text{target present}).$$

The unconditional probability that the target is at a particular position i is

$$P(i) = P(\text{target present})P(i \mid \text{target present}).$$

Therefore,

$$P(i) = \frac{1}{2} \cdot \frac{1}{n}.$$

Hence,

$$P(i) = \frac{1}{2n}.$$

Now let C_i be the cost when the target is at position i , and let C_F be the cost when the target is absent.

The expected cost is

$$E_2 = \sum_{i=1}^n P(i)C_i + P(\text{absent})C_F.$$

Substituting the probabilities,

$$E_2 = \sum_{i=1}^n \frac{1}{2n} C_i + \frac{1}{2} C_F.$$

Factoring out $\frac{1}{2n}$,

$$E_2 = \frac{1}{2n} (C_1 + C_2 + \cdots + C_n) + \frac{1}{2} C_F.$$

Now define the average successful-search cost as

$$\overline{C_S} = \frac{C_1 + C_2 + \cdots + C_n}{n}.$$

Therefore,

$$\frac{1}{2n} (C_1 + C_2 + \cdots + C_n) = \frac{1}{2} \overline{C_S}.$$

Hence,

$$E_2 = \frac{1}{2} \overline{C_S} + \frac{1}{2} C_F$$

or equivalently,

$$E_2 = \frac{\overline{C_S} + C_F}{2}.$$

Comparison of the Two Probability Situations

The two situations do not have the same average running time.

In the first situation, the target is equally likely to be at any one of the n positions in the array, and the probability that the target is not in the array is the same as the probability of each individual position. Therefore, the probability of an unsuccessful search is only

$$\frac{1}{n+1}.$$

As the size of the array increases, this probability becomes very small. This means that most searches in this situation are successful searches.

In the second situation, the probability that the target is in the array is the same as the probability that it is not in the array. Therefore, an unsuccessful search occurs with probability

$$\frac{1}{2}.$$

When the target is present, each position is equally likely, but half of all searches are still unsuccessful.

An unsuccessful Binary Search usually requires searching until the interval is empty or nearly empty, so it generally costs more than an average successful search. Because the second situation gives a much larger probability to the unsuccessful-search case, its average running time is higher than that of the first situation.

Therefore, the second situation has the higher average running time.

However, the difference is only in the constant part of the average cost. Both situations still have logarithmic growth because Binary Search reduces the remaining search interval approximately by half after each comparison. Thus, both average running times are

$$O(\lg n).$$

Task 3

Average Cost and Worst-Case Cost

The worst-case cost of Binary Search is indeed greater than or equal to the average cost. However, this does not mean that they must have different asymptotic complexities.

Binary Search reduces the remaining search interval approximately by half after every comparison. Therefore, the longest possible search path requires approximately

$$\lg n$$

comparisons. Hence, the worst-case cost is

$$C_{\text{worst}}(n) = \Theta(\lg n).$$

For the average case, some target values are found earlier than the worst case. However, most elements in the Binary Search decision tree are located near the lower levels of the tree. In a perfectly balanced tree, approximately half of the elements are at the deepest level, one-fourth are one level above it, one-eighth are two levels above it, and so on.

Therefore, although some searches require fewer comparisons, a large fraction of the possible searches still require close to $\lg n$ comparisons. Consequently, the average number of comparisons also grows proportionally to $\lg n$.

The average cost may be slightly smaller than the worst-case cost. For example, their costs may behave approximately like

$$C_{\text{average}}(n) = \lg n - c$$

and

$$C_{\text{worst}}(n) = \lg n,$$

where c is a constant.

Thus, the worst-case cost is still numerically larger, but subtracting a constant does not change the asymptotic growth rate. Therefore,

$$C_{\text{average}}(n) = \Theta(\lg n)$$

and

$$C_{\text{worst}}(n) = \Theta(\lg n).$$

Hence, there is no contradiction in both the average cost and the worst-case cost being $O(\lg n)$. The worst case is higher in exact cost, but both grow at the same logarithmic rate as the input size increases.

Task 4

Experimental Estimation of the Average Cost

To experimentally estimate the average cost of Binary Search, I used the number of middle elements examined as the cost indicator. Each iteration of Binary Search examines one middle element, so the cost counter is increased once per iteration.

The experiment was performed using sorted arrays of the following sizes:

100, 500, 1000, 1500, 3000, 6000.

For each input size, multiple randomly generated arrays were tested and the arithmetic mean of the measured costs was calculated. For a successful search, all elements of the array were searched and their average cost was calculated. For an unsuccessful search, multiple randomly generated target values that were not present in the array were tested and their average cost was calculated.

The following quantities were measured:

- **avg_success**: average number of comparisons when the target is present in the array.
- **avg_not_found**: average number of comparisons when the target is not present in the array.
- **Situation 1**: the expected cost when each of the n successful positions and the one unsuccessful case has probability $1/(n+1)$.
- **Situation 2**: the expected cost when the probability that the target is present is $1/2$, the probability that it is absent is $1/2$, and every array position is equally likely when the target is present.

Experimental Results

The obtained results are shown in Table 1.

Figure 1 shows the experimental average costs for the different input sizes together with the theoretical logarithmic function $\lg n$.

The graph shows that all measured costs increase slowly as the input size increases. This behavior is consistent with logarithmic growth.

Table 1: Experimental Average Cost of Binary Search

n	Avg. Success	Avg. Not Found	Situation 1	Situation 2	$\lg n$
100	5.8000	6.7310	5.8092	6.2655	6.6439
500	7.9960	8.9775	7.9980	8.4868	8.9658
1000	8.9870	9.9790	8.9880	9.4830	9.9658
1500	9.6427	10.6650	9.6433	10.1538	10.5507
3000	10.6390	11.6245	10.6393	11.1317	11.5507
6000	11.6370	12.6305	11.6372	12.1338	12.5507

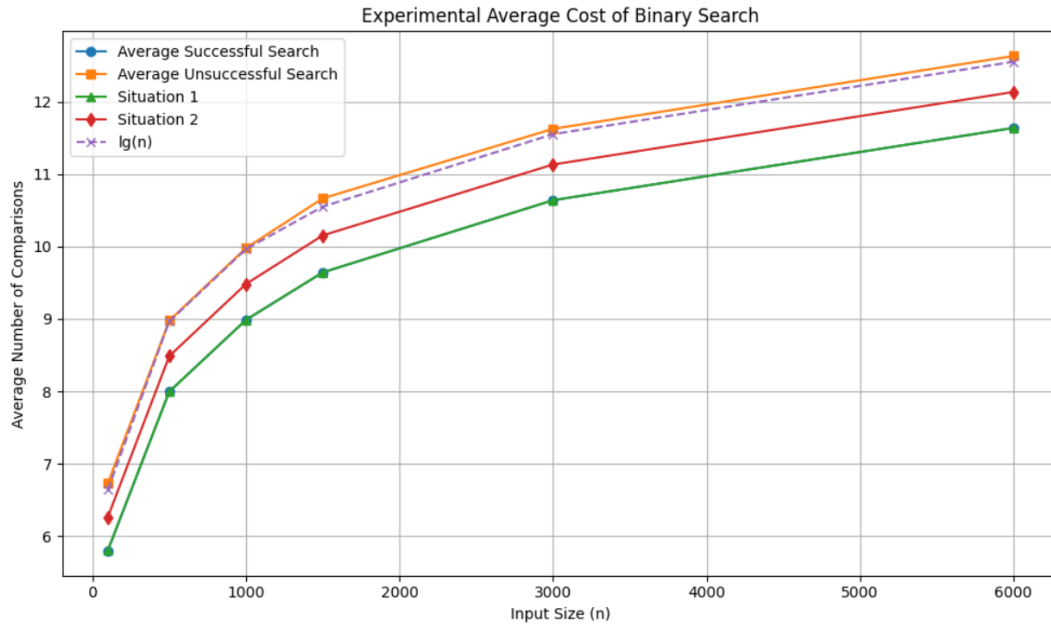


Figure 1: Experimental average costs of Binary Search for different input sizes compared with the theoretical logarithmic growth $\lg n$.

Discussion of the Results

The experimental results are consistent with the theoretical conclusion that the average cost of Binary Search is $O(\lg n)$.

As the array size increases, the number of comparisons increases very slowly. For example, when the input size doubles from $n = 1500$ to $n = 3000$, the average successful-search cost increases from

$$9.6427 \text{ to } 10.6390.$$

The increase is approximately

$$10.6390 - 9.6427 = 0.9963.$$

Similarly, when the input size doubles again from 3000 to 6000, the average successful-search cost increases from

$$10.6390 \text{ to } 11.6370$$

which is an increase of approximately

$$11.6370 - 10.6390 = 0.9980.$$

This agrees closely with logarithmic growth because

$$\lg(2n) = \lg n + 1.$$

Therefore, when the input size doubles, a logarithmic algorithm is expected to require approximately one additional level of searching.

The **unsuccessful-search results** show similar behavior. For example, the average unsuccessful-search cost increases from 10.6650 for $n = 1500$ to 11.6245 for $n = 3000$, and then to 12.6305 for $n = 6000$.

Thus, the unsuccessful-search cost also grows approximately logarithmically.

Successful and Unsuccessful Searches

The experimental results show that an unsuccessful search generally requires more comparisons than an average successful search.

For example, at

$$n = 6000,$$

the average successful-search cost is

$$11.6370,$$

while the average unsuccessful-search cost is

$$12.6305.$$

The reason is that a successful search may terminate at an earlier level of the Binary Search decision tree when the target is found. In contrast, an unsuccessful search normally continues until the search interval becomes empty.

However, both costs still grow at the same logarithmic rate.

Comparison of Situation 1 and Situation 2

The experimental results also support the conclusion from Task 2 that Situation 2 has a higher average cost than Situation 1.

For example, for

$$n = 1000,$$

the measured average costs are

$$E_1 = 8.9880$$

and

$$E_2 = 9.4830.$$

Similarly, for

$$n = 6000,$$

the measured values are

$$E_1 = 11.6372$$

and

$$E_2 = 12.1338.$$

In Situation 1, the probability of an unsuccessful search is only

$$\frac{1}{n+1}.$$

As n becomes large, this probability becomes very small. Therefore, the average cost in Situation 1 becomes very close to the average successful-search cost.

This behavior can be seen directly from the experiment. For $n = 6000$,

$$\text{Avg. Success} = 11.6370$$

and

$$\text{Situation 1} = 11.6372.$$

The two values are almost identical because the unsuccessful-search case has only a very small probability.

In Situation 2, however, the probability of an unsuccessful search is always

$$\frac{1}{2}.$$

Since unsuccessful searches generally require more comparisons, assigning half of the probability to them increases the expected cost. Therefore, Situation 2 consistently produces a higher average cost than Situation 1.

Comparison with $\lg n$

The measured costs do not need to be exactly equal to $\lg n$. The theoretical complexity $O(\lg n)$ describes the rate of growth rather than the exact number of comparisons.

For example, at

$$n = 6000,$$

we have

$$\lg(6000) \approx 12.5507,$$

while the average successful-search cost is

$$11.6370.$$

The average successful cost is lower because some targets are found before the deepest level of the Binary Search decision tree.

The important observation is that both the theoretical value and the measured average costs increase at approximately the same logarithmic rate as n increases.

Therefore, the experimental results support the theoretical conclusions

$$E_1(n) = O(\lg n)$$

and

$$E_2(n) = O(\lg n).$$

Overall, the experiment confirms that the average running cost of Binary Search grows logarithmically. It also confirms that Situation 2 has a higher average cost than Situation 1 because unsuccessful searches receive a much larger probability in Situation 2.

AI use Acknowledgment

Generative AI (ChatGPT) was used to assist with LaTeX formatting and presentation of this report.